



**Hewlett Packard
Enterprise**



Career Preview Program - 3 Project

Final Presentation

July 3, 2026



**Hewlett Packard
Enterprise**



PROJECT TITLE:

**AI-ENABLED BUILD WORKFLOW FOR INFRASTRUCTURE
DEPLOYMENT**

PROJECT ID: CPP3 - 76 (Hybrid Cloud and OCTO)

PROJECT NAME: AI-ENABLED BUILD WORKFLOW FOR
INFRASTRUCTURE DEPLOYMENT

COLLEGE NAME: RV COLLEGE OF ENGINEERING

PROJECT TEAM

Sl. No	NAME	EMAIL
1	NISHANT V H	nishantvh.cy23@rvce.edu.in
2	JANANI S	jananisrikanth37@gmail.com
3	TEJASVI VASANT HEGDE	tejasvihegde303@gmail.com
4	CHETOHAAR J M	chethohaarjm@gmail.com
5	AKSHITH DESU	akshith.desu@gmail.com
6	NIKITHA RODRIGUES	nikitharodrigues3@gmail.com

HPE MENTOR: Aditya Abhay Nitsure (aditya-abhay.nitsure@hpe.com)

GitHub Link for the Project:

<https://github.com/kernelops/hpe-pcai-ai-workflow>





**Hewlett Packard
Enterprise**

Problem Statement



HPE Private Cloud AI (PCAI) is deployed on HPE hardware through automated factory build workflows involving **servers, storage, networking, OS, iLO, NFS, MinIO**, and software stack configuration. Since these workflows contain many **dependent tasks**, errors can occur due to wrong software versions, last-minute stack changes, incorrect configuration procedures, or incomplete execution of earlier steps.

When such **failures** happen, engineers must **manually analyze logs**, identify the root cause, fix the issue, and either resume the workflow or **restart it** from the beginning. This manual recovery process is **time-consuming** and **inconsistent**.

This project aims to build an **AI-enabled infrastructure deployment workflow using Airflow with LLM/RAG-based remediation**. The system executes deployment tasks, understand errors and warnings, pinpoint failure causes, apply fixes, resume execution, and **complete the workflow automatically**.

Solution Proposal

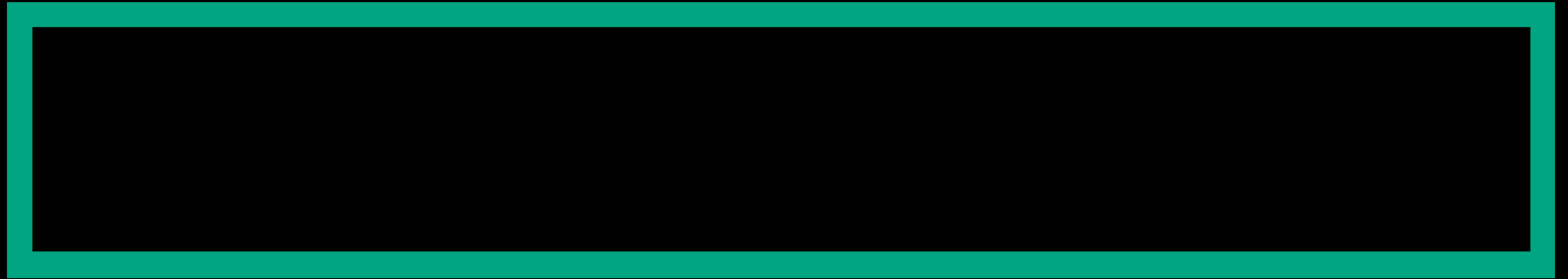
1. Closed-Loop Hybrid Autofix Pipeline

2. Dual-Telemetry Ingestion

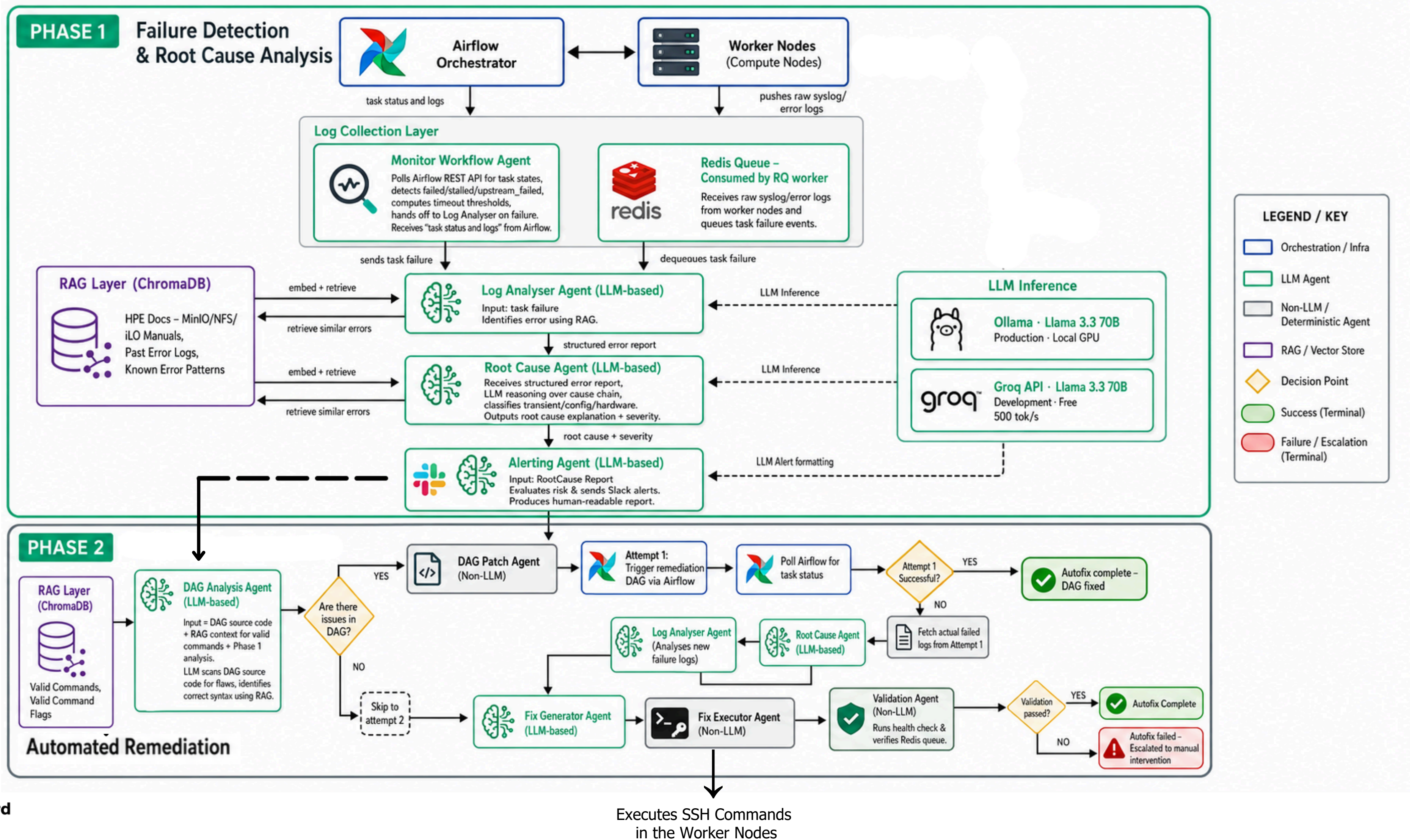
- Active Monitor Agent: Continuously polls the Airflow REST API to capture application-level task failures.
- Passive Redis Queue Ingest: Captures raw system/kernel level logs pushed directly from worker node telemetry agents.

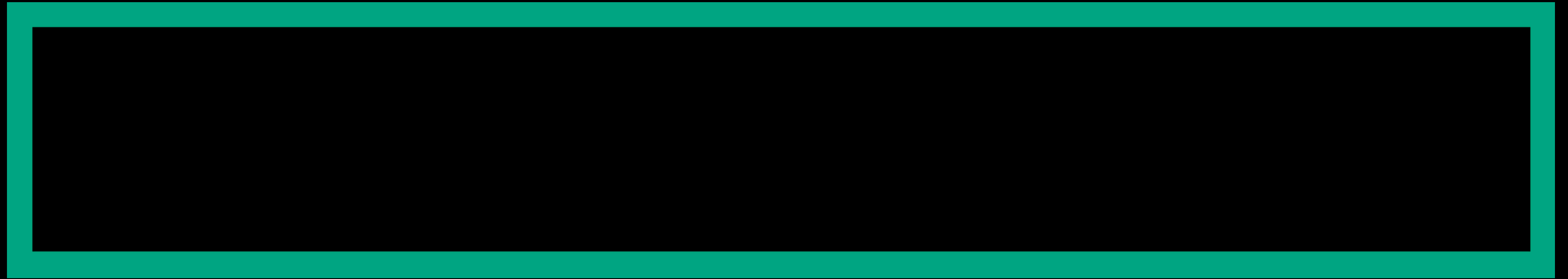
3. Two-Attempt Intelligent Healing Flow

- a. Attempt 1 (DAG Source Patching): LLM analyzes orchestration scripts to dynamically write and execute a corrected `remediation_workflow.py` for logical/configuration flaws.
 - b. Attempt 2 (SSH Infrastructure Healing): If Attempt 1 fails, the system fetches fresh logs, generates system-level bash commands, and uses Paramiko SSH to heal the node's physical configuration (with secure sudo injection).
- ## 4. Post-Fix Validation:
- Validates if the autofix was successful. If not, it generates a summary and raises the issue for manual intervention.



SYSTEM ARCHITECTURE





DEPLOYMENT WORKFLOW AND THE AGENTIC PIPELINE

DEPLOYMENT WORKFLOW

Run Broken DAG

Run Healthy / Error-free DAG

 **AI Enabled Build Workflow**
for Infrastructure Deployment

DEV

AD

Admin
Platform Owner

Dashboard

Worker Nodes

SSH Validation

Deployment

Log Insights

Agent Ops

LIVE PULSE

100% Connected

Total
1

Unreachable
0

Deployment: Idle

Airflow Deployment

This will trigger an Airflow DAG that deploys applications across all reachable worker nodes. The DAG will execute SSH commands on each node for system updates, Docker installation, and application deployment.

Run Deployment (Broken)

Run Deployment (Healthy)

CURRENT RUN ID	DAG	STATUS	LAST UPDATED	TASK STREAMS
-	deployment_workflow	IDLE	-	0

Workflow Monitor

Copy Logs

Auto-scroll: On

Clear Logs

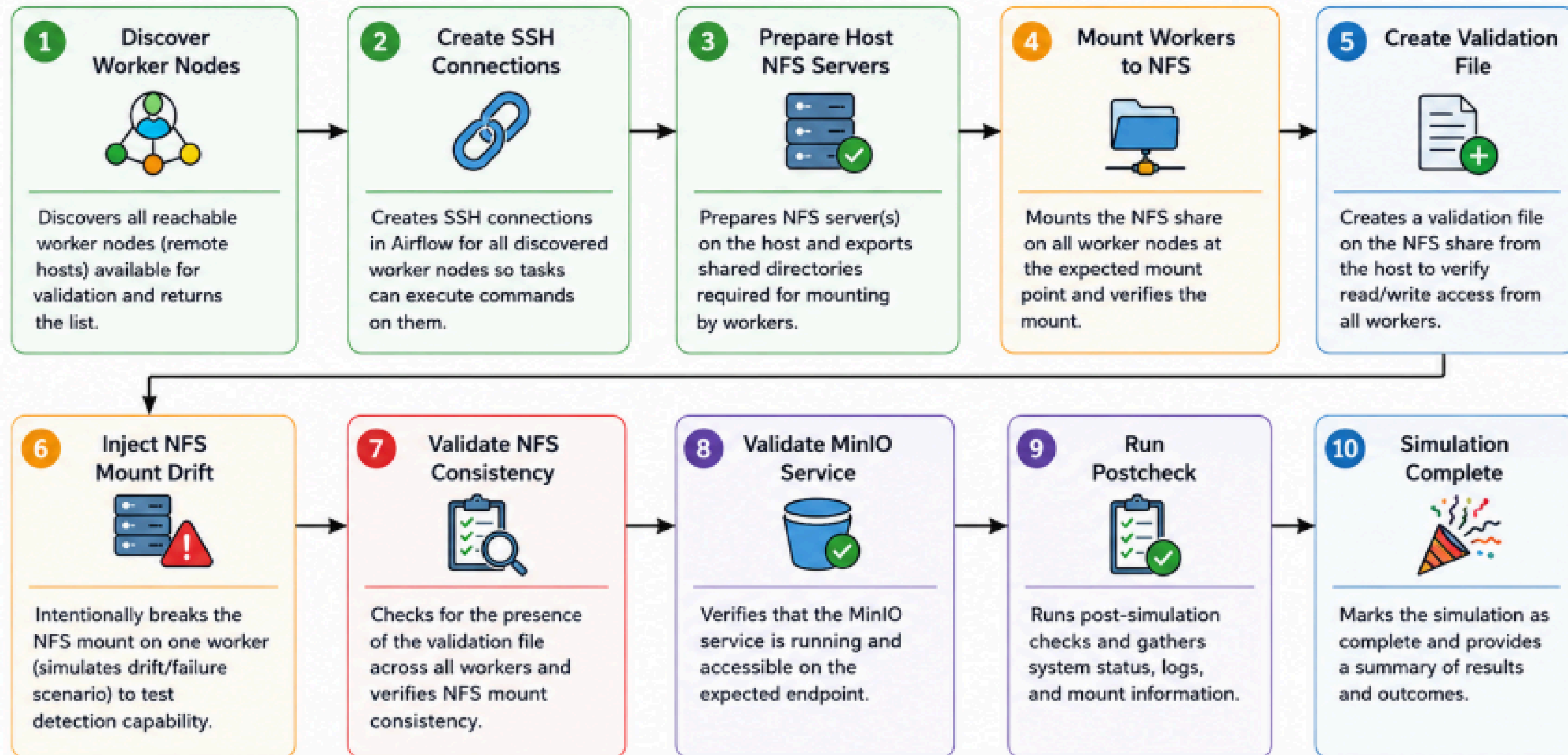
Status: Idle

No logs available. Start a deployment to see real-time logs from Airflow.



Hewlett Packard
Enterprise

NFS INCONSISTENCY DAG-ENVIRONMENTAL DRIFT SCENARIO



1. Real Folders on Each Worker

Worker 1
192.168.56.101



/srv/nfs_a/shared
(Real Folder)

Worker 2
192.168.56.102



/srv/nfs_b/shared
(Real Folder)

Each worker has one real folder that is **exported** over the network via NFS.

2. Two NFS Backends Available

NFS A

192.168.56.101
:/srv/nfs_a/shared



NFS B

192.168.56.102
:/srv/nfs_b/shared



These exports become two NFS backends accessible over the network.

3. Initial State – Both Workers Mount NFS A

Worker 1
192.168.56.101



/mnt/nfs

NFS A

192.168.56.101:/srv/nfs_a/shared

Both point to
the same NFS A

Worker 2
192.168.56.102



/mnt/nfs

NFS A

192.168.56.101:/srv/nfs_a/shared

Now both workers are looking at the same shared folder (NFS A).

4. Worker 1 Writes a File

On Worker 1

```
echo "deployment-check" > /mnt/nfs/check.txt
```

NFS A

192.168.56.101:/srv/nfs_a/shared



check.txt
deployment-check

Because Worker 2 also
mounts NFS A, it can
read the file.

On Worker 2

```
cat /mnt/nfs/check.txt
```

Output:
deployment-check

File is created on NFS A and visible to both workers.

5. Introduce Inconsistency – Worker 2 Switched to NFS B

Worker 1
192.168.56.101



/mnt/nfs

NFS A

192.168.56.101:/srv/nfs_a/shared

Now they point to
different backends!

On Worker 2

```
cat /mnt/nfs/check.txt
```

Output:
check.txt missing

Worker 2
192.168.56.102



/mnt/nfs

NFS B

192.168.56.102:/srv/nfs_b/shared

Worker 2 is checking NFS B, but the file exists on NFS A. Validation fails.

6A. Attempt 1 – Fix the Symptom (Not the Root Cause)

On Worker 2

```
echo "deployment-check" > /mnt/nfs/check.txt
```

NFS B

192.168.56.102:/srv/nfs_b/shared



check.txt
deployment-check

Mount Source Check

Expected: NFS A

192.168.56.101:/srv/nfs_a/shared

Actual: NFS B

192.168.56.102:/srv/nfs_b/shared

✗ Attempt 1 Fails
Reason: Wrong NFS backend

File exists now, but worker is still on the wrong NFS backend.

6B. Attempt 2 – Fix the Root Cause (Correct the Environment)

On Worker 2

```
sudo umount -lf /mnt/nfs  
sudo mkdir -p /mnt/nfs  
sudo mount 192.168.56.101:/srv/nfs_a/shared /mnt/nfs
```

/mnt/nfs

NFS A

192.168.56.101:/srv/nfs_a/shared

Worker 2 is mounted back to NFS A.

6C. Final Validation – Everything is Consistent Again

Worker 1
192.168.56.101



/mnt/nfs

NFS A

192.168.56.101:/srv/nfs_a/shared

Both back on
NFS A

On Worker 2

```
cat /mnt/nfs/check.txt
```

Output:
deployment-check

Worker 2
192.168.56.102



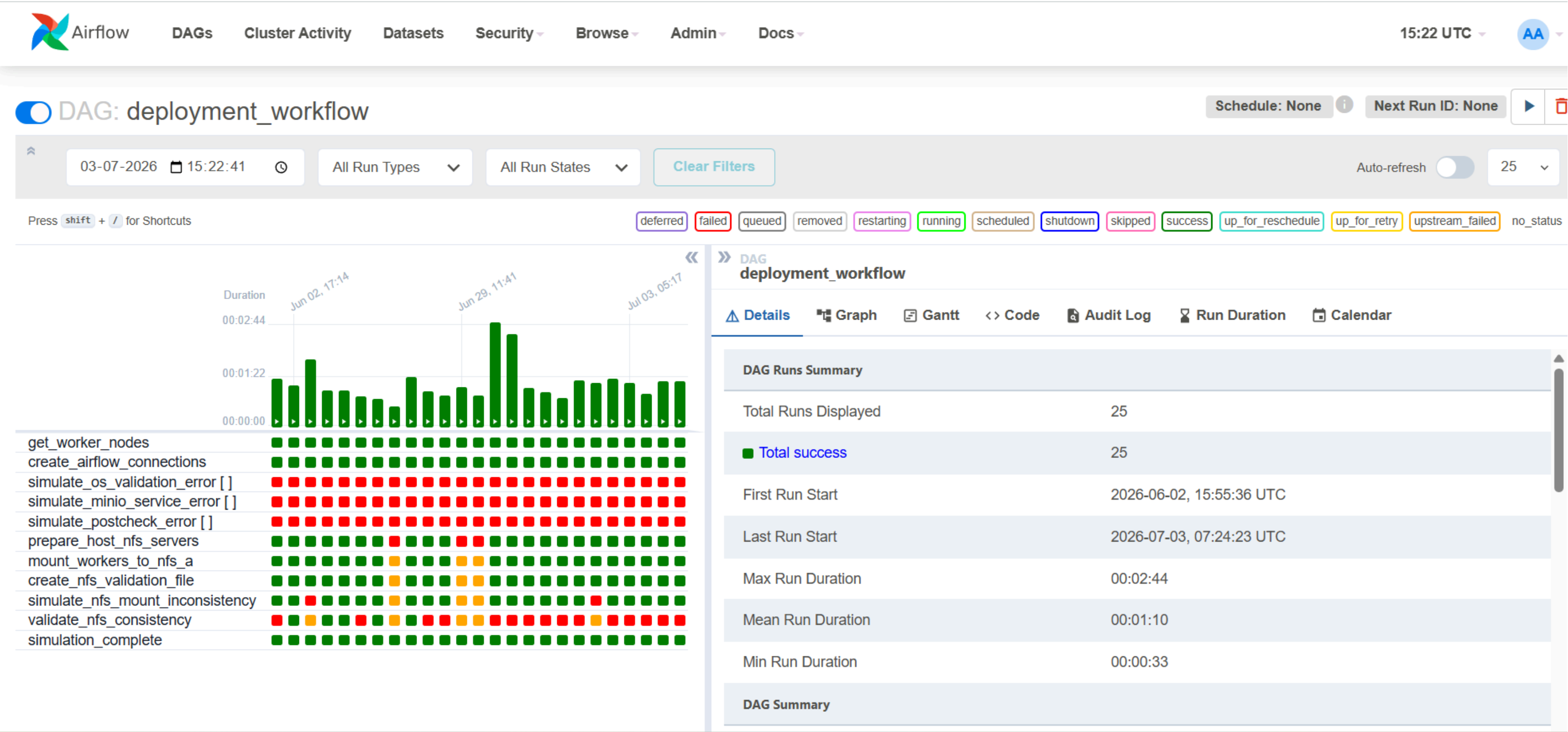
/mnt/nfs

NFS A

192.168.56.101:/srv/nfs_a/shared

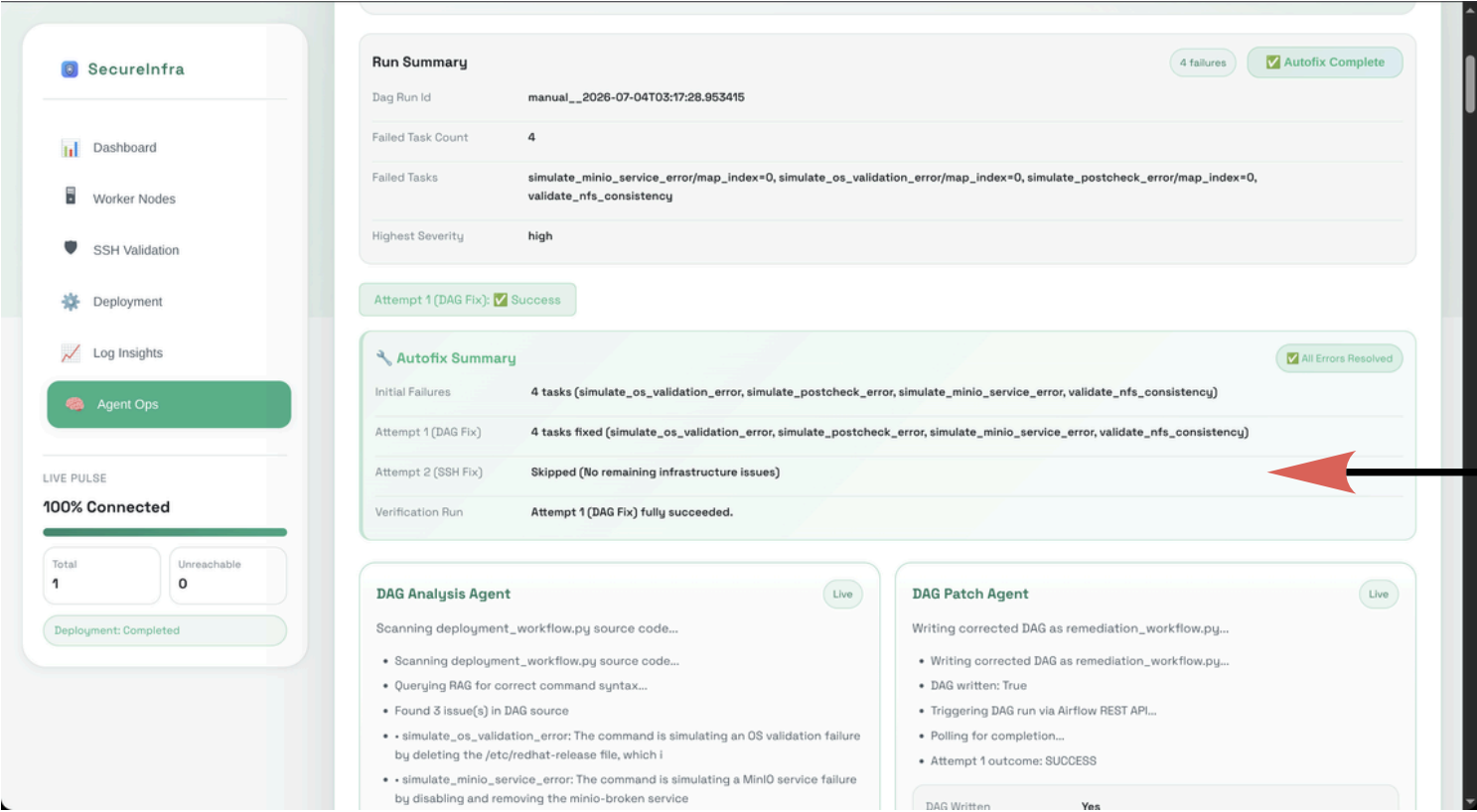
Both workers see the same file again. Environment issue is fixed.

NFS INCONSISTENCY DAG-ENVIRONMENTAL DRIFT SCENARIO



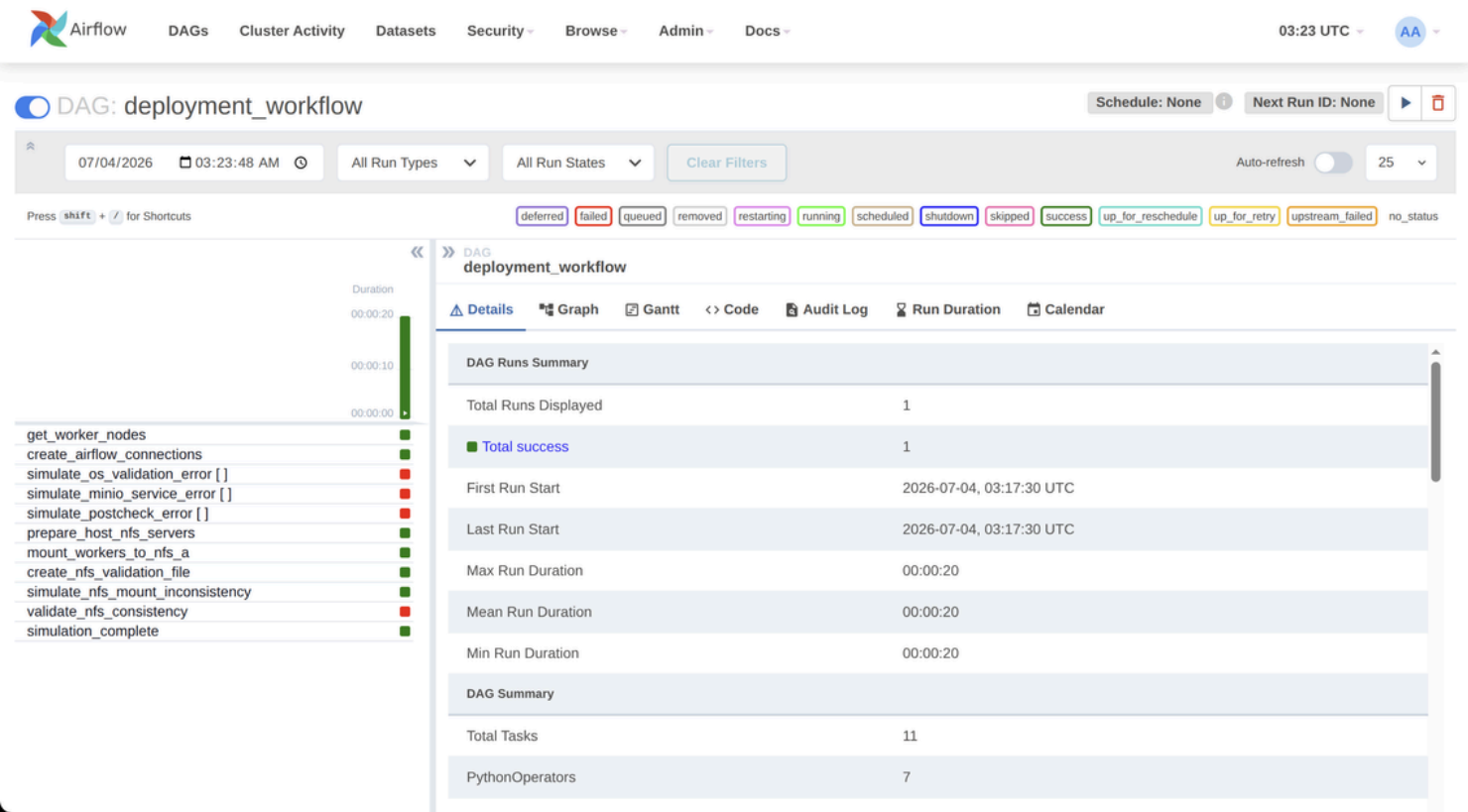
AGENT OPS PANEL - FIXING THE DAG ISSUES IN ATTEMPT 1

AGENTS OPS
PANEL

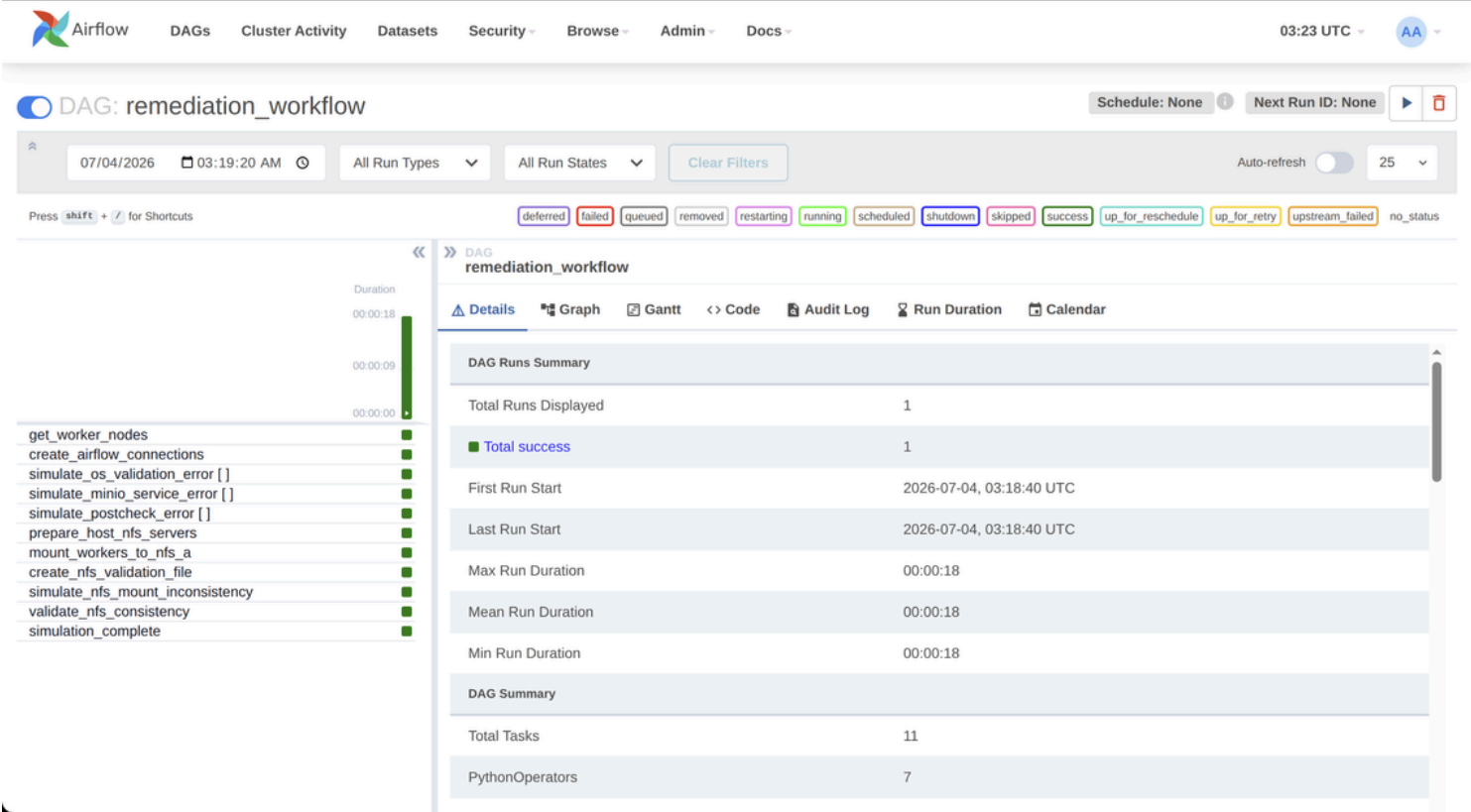


Autofix
summary

deployment_workflow

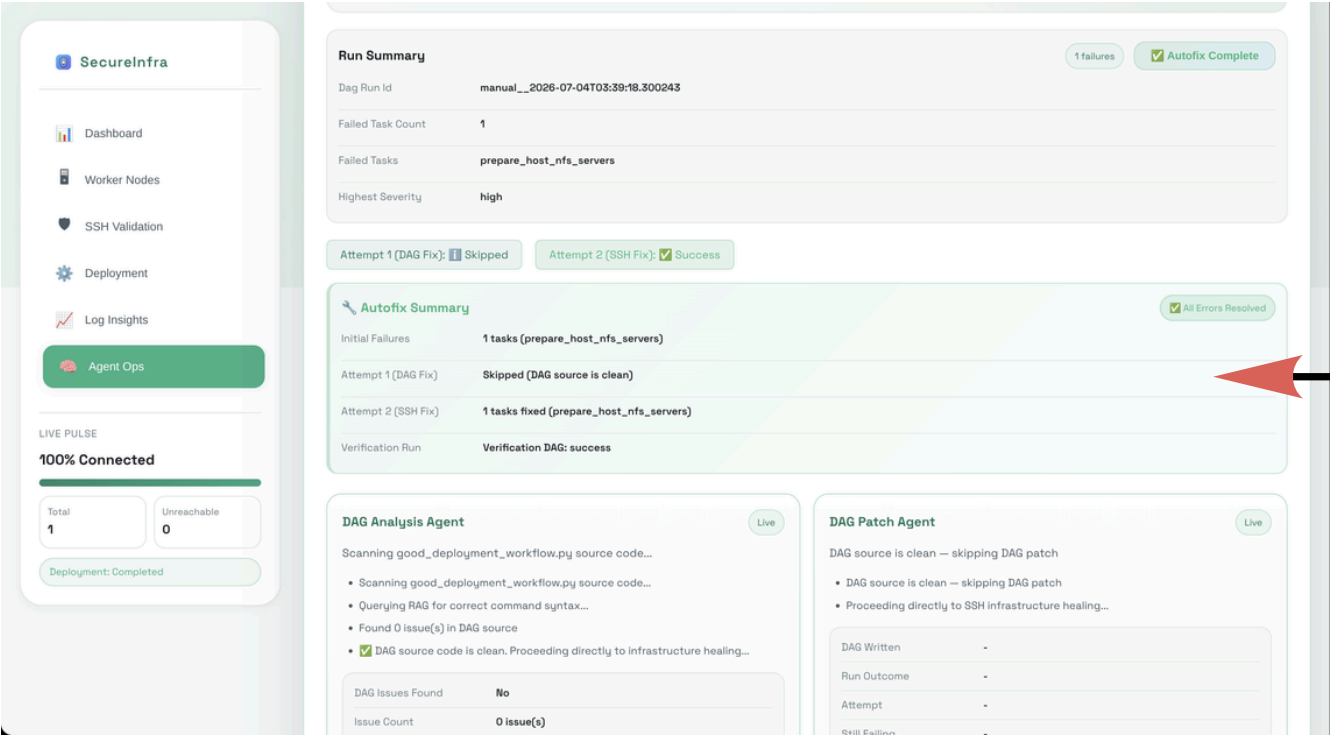


remediation_workflow



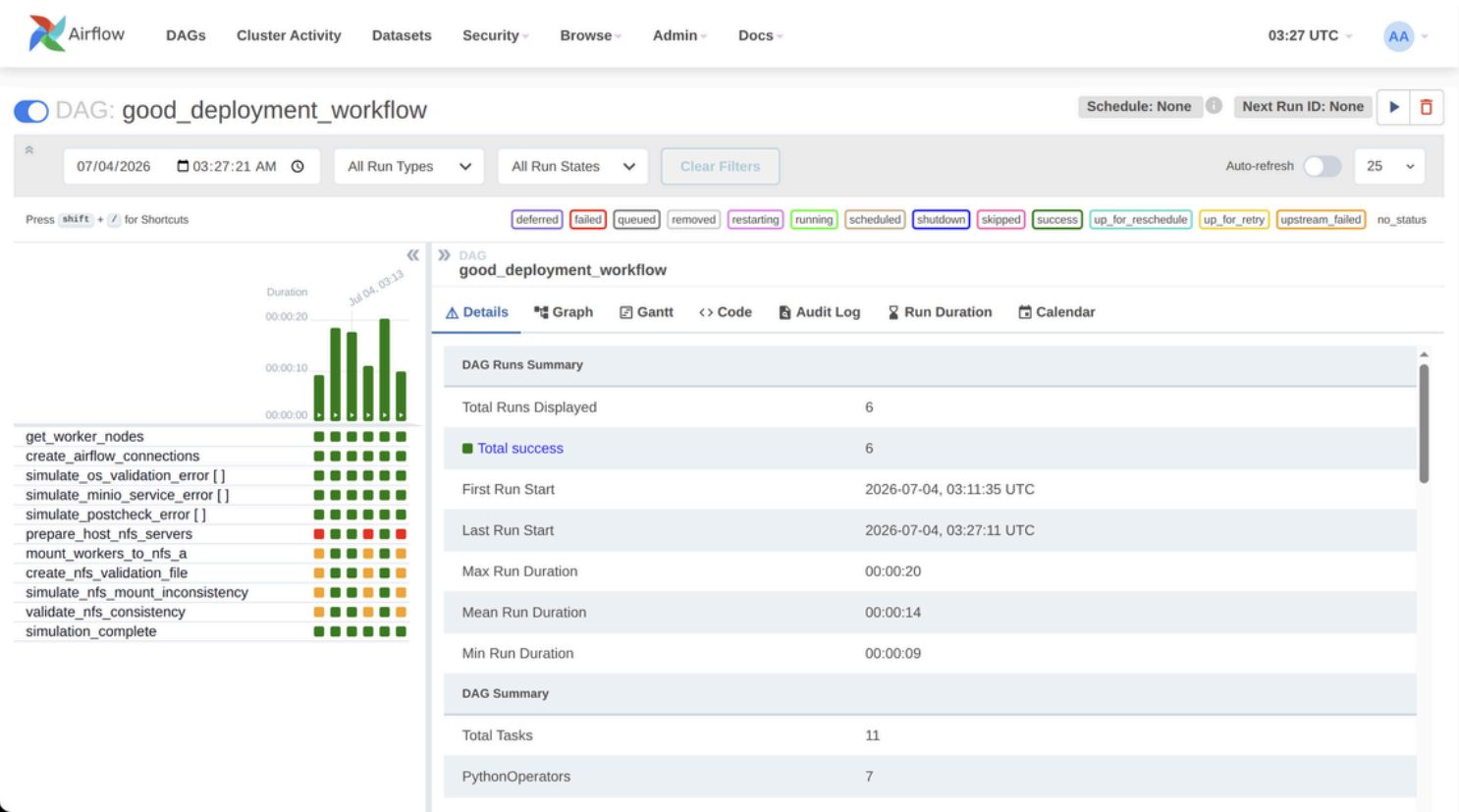
AGENT OPS PANEL - FIXING THE ENVIRONMENTAL ISSUES IN ATTEMPT 2

AGENTS OPS
PANEL

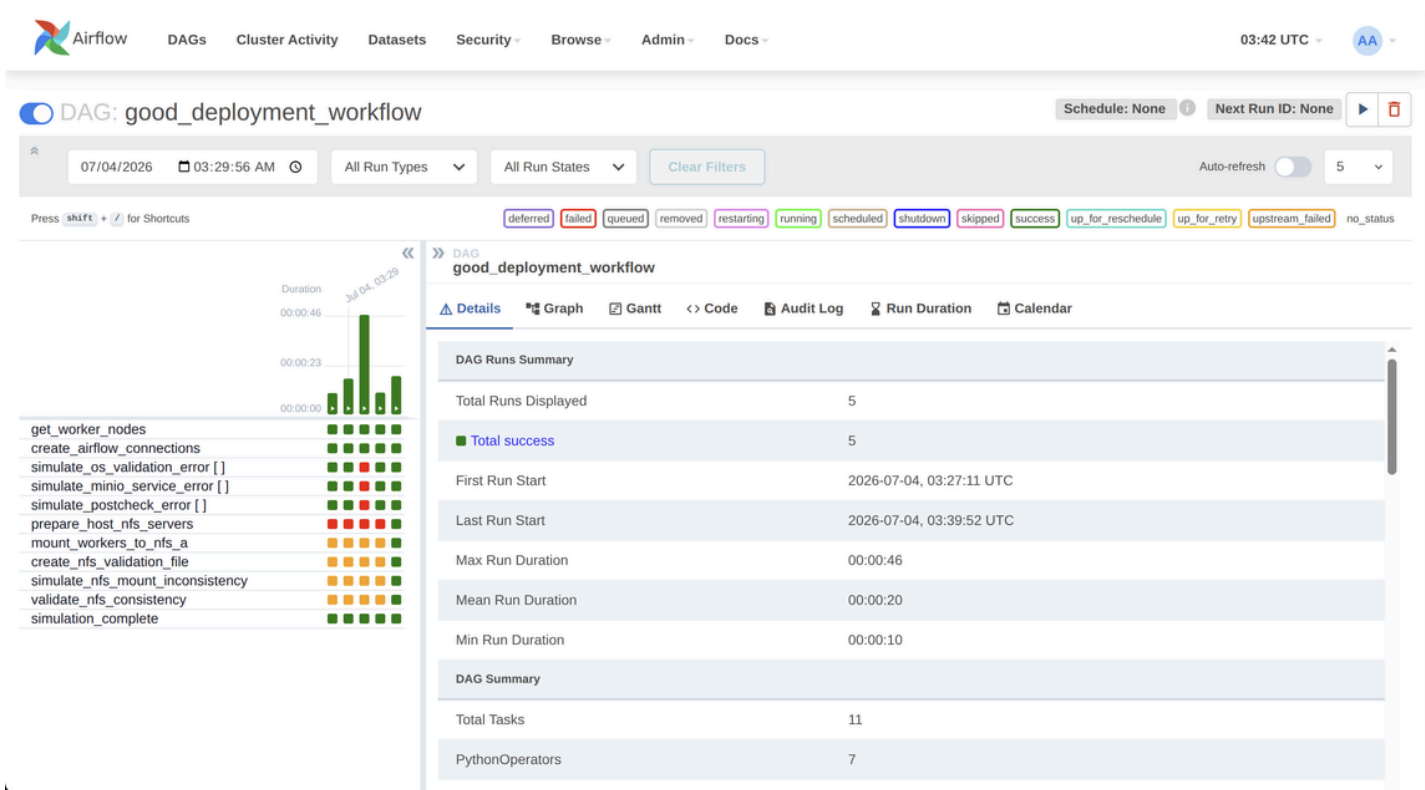


Autofix
summary

good_deployment_workflow BEFORE THE FIX



good_deployment_workflow AFTER THE FIX



Orchestration Frameworks: CrewAI vs. LangGraph

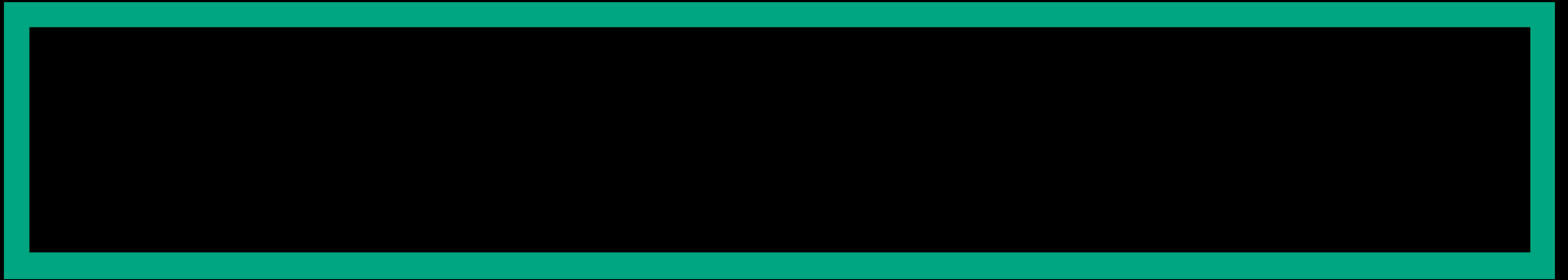
CrewAI	LangGraph (Our Choice)
Autonomous role-based agents	Stateful workflow graph
Flexible conversations	Deterministic execution
Dynamic task planning	Explicit routing & loops
Best for creative/open-ended tasks	Best for enterprise workflows
Hard to control execution	Full control over execution

Agents Summary

No.	Agent	Type
1	Run Workflow	Non-LLM
2	Monitor Workflow	Non-LLM
3	Log Analyzer	LLM
4	Root Cause	LLM
5	Alerting	LLM
6	DAG Analysis	LLM
7	DAG Patch	Non-LLM
8	Fix Generator	LLM
9	Fix Executor	Non-LLM
10	Validation	Non-LLM

Agent LLM Call Summary

Scenario	Use Case	LLM calls	Agents in order
Failure Detected → Autofix Disabled	Manual Intervention Mode: The system encounters an error but is configured to only analyze the issue and notify engineers without attempting to fix it automatically.	3	Log Analyser Root Cause Alerting
Failure Detected → Autofix Enabled → DAG Issue Found → Patch Succeeds	Pipeline Code Error: There is a typo or logic error in the Airflow DAG file itself. The AI finds the bug, applies the code patch, and the pipeline resumes successfully.	4	Log Analyser Root Cause Alerting DAG Analysis
Failure Detected → Autofix Enabled → DAG Issue Found → Patch Fails → Environment Fix Triggered	Complex / Cascading Failure: The AI tries to fix a pipeline issue, but the fix fails (e.g., requiring a missing dependency on the server). It loops back, analyzes the new failure, and generates SSH commands to fix the underlying server issue.	7 8+ (if iterative Autofix)	Log Analyser Root Cause Alerting DAG Analysis Log Analyser Root Cause Fix Generator (2+ calls if iterative Autofix)
Failure Detected → Autofix Enabled → DAG is Healthy → Environment Fix Triggered	Environment/System Error: The pipeline code is perfectly fine, so the failure is environmental (e.g., server out of disk space, missing OS library). The AI directly generates a fix for the worker node.	5 6+ (if iterative Autofix)	Log Analyser Root Cause Alerting DAG Analysis Fix Generator (2+ calls if iterative Autofix)



RAG LAYER

RAG with ChromaDB

- Private servers and **private clouds** run internal tools, custom configs, and **homegrown** error messages that **no public LLM** was ever trained on.
- An LLM alone would just guess or **hallucinate** a fix.
- RAG solves this by **storing real past errors** and their **proven fixes** in a knowledge base, so the system retrieves an actual documented solution instead of making one up.
- This makes the pipeline **accurate, explainable**, and easy to keep updating — every new fix just gets added to the KB.

Why ChromaDB?

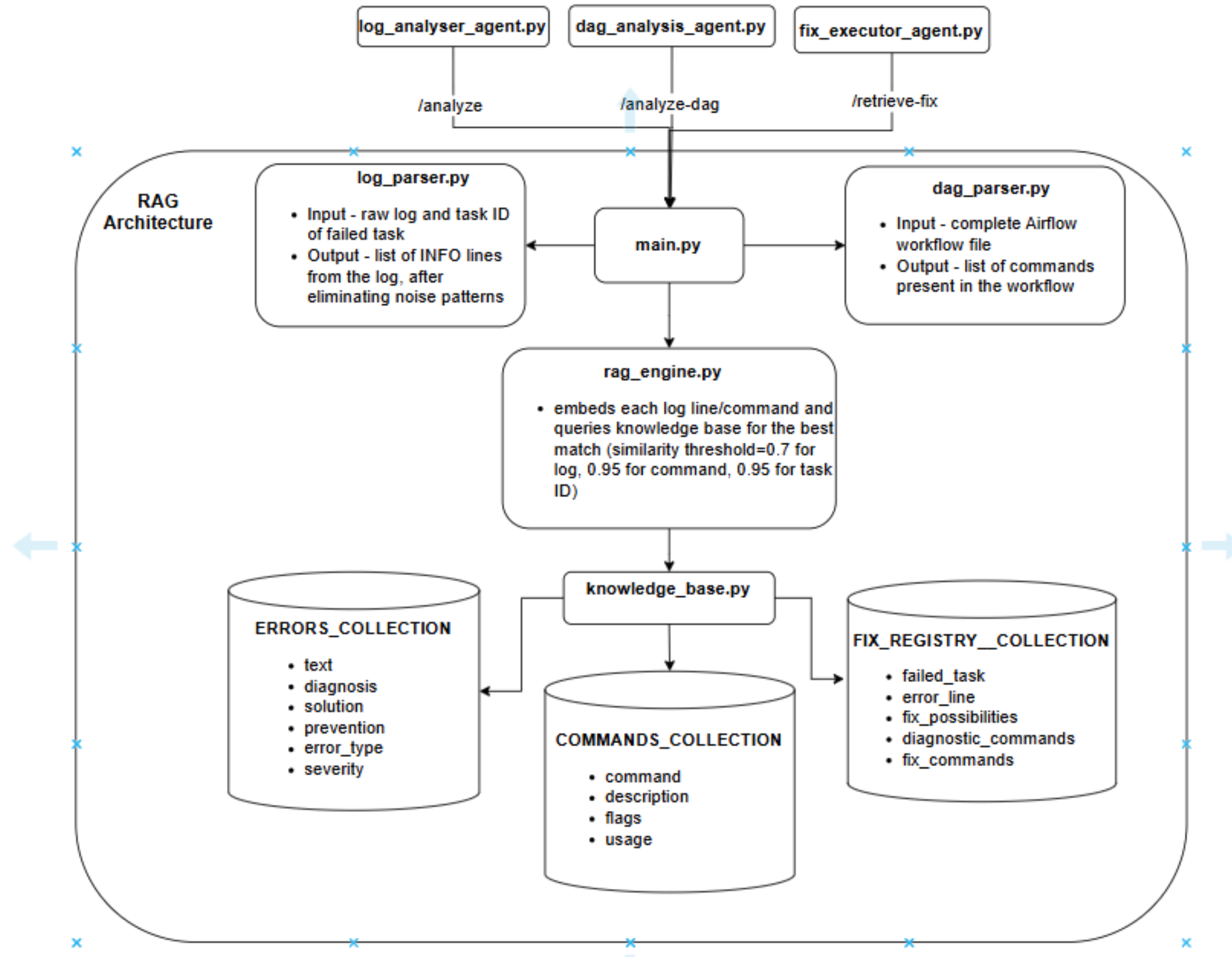
- ChromaDB is a **lightweight, open-source vector database** that runs locally.
- It is designed to store embeddings and perform **fast similarity searches**. The knowledge base is saved and does not need to be **rebuilt every time**.
- ChromaDB is easy to set up and integrates seamlessly with Python, **requiring minimal code** and no additional infrastructure.

RAG with ChromaDB

Cosine Similarity

- Error logs vary in **length and wording**, but cosine similarity cares about the direction of the vectors, not their magnitude — so it compares meaning, not sentence length.
- This makes it reliable even when a new error log is worded slightly differently from the one stored in the KB.
- It's the standard, **well-tested metric** for text/semantic similarity search, and ChromaDB supports it natively (hnsw:space: cosine).

RAG Workflow



RAG Knowledge Base Contents

ERRORS_COLLECTION

```
{
  "id": "err_001",
  "text": "wget: unable to resolve host address. ",
  "source": "MinIO Installation Logs",
  "diagnosis": "The system cannot resolve the hostname to an IP address. This indicates a DNS resolution failure. Possible underlying issues are no internet connection, incorrect DNS settings, or a blocked DNS port. Check network connectivity and DNS resolution with ping (e.g. ping google.com). Edit /etc/resolv.conf and add reliable DNS servers (Put nameserver 8.8.8.8).",
  "solution": "Check network connectivity and DNS resolution with ping (e.g. ping google.com). Edit /etc/resolv.conf and add reliable DNS servers (Put nameserver 8.8.8.8).",
  "prevention": "Ensure stable network connection. Configure stable DNS servers. Monitor DNS resolution regularly. Verify firewall settings do not block DNS traffic.",
  "error_type": "Network configuration error",
  "severity": "Medium",
  "retrieved_sources": "https://stackoverflow.com/questions/24821521/wget-unable-to-resolve-host-address-http"
},
```

COMMANDS_COLLECTION

```
{
  "id": "cmd_020",
  "command": "virsh destroy",
  "description": "Immediately terminates the specified domain",
  "flags": "-c, --connect <URI>, -d, --debug <LEVEL>, -e, --escape <STRING>, -h, --help, -k, --keepalive-interval <INTERVAL>, -K, --keepalive-count, -l, --log <FILE>",
  "usage": "virsh destroy domain [--graceful] [--remove-logs]",
  "source": "https://www.libvirt.org/manpages/virsh.html#description"
},
```

RAG Knowledge Base Contents

FIX_REGISTRY_COLLECTION

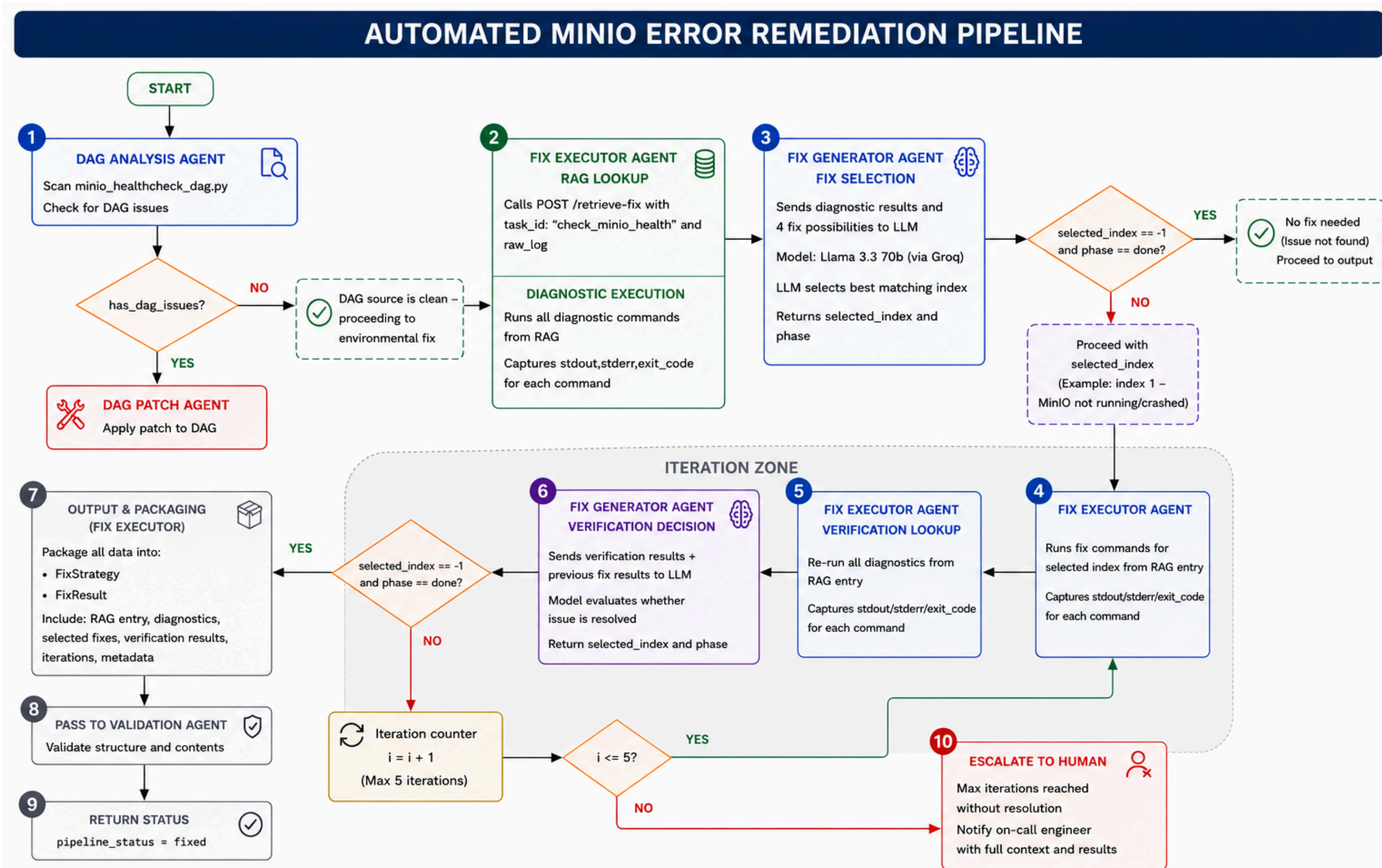
```
FIX_REGISTRY = [  
  {  
    "id": "fix_001",  
    "failed_task": "check_minio_health",  
    "error_line":  
    [  
      "SSH operator error: exit status = 7",  
      "curl: (7) Failed to connect to localhost port 9000 after 2 ms: Could not connect to server"  
    ],  
    "fix_possibilities": ["MinIO not installed", "MinIO not running/crashed", "MinIO not on the right port", "Firewall blocking MinIO port"],  
    "diagnostic_commands": [  
      [  
        "which minio",  
        "which mc"  
      ],  
      [  
        "ps aux | grep -v grep | grep minio",  
        "sudo systemctl is-active minio",  
        "sudo systemctl status minio --no-pager"  
      ],  
      [  
        "ss -tuln | grep 9000",  
        "ss -tuln | grep -E '900[0-9]'"  
      ],  
      [  
        "sudo ufw status",  
        "sudo iptables -L INPUT -n | head -10"  
      ]  
    ],  
  },  
]
```

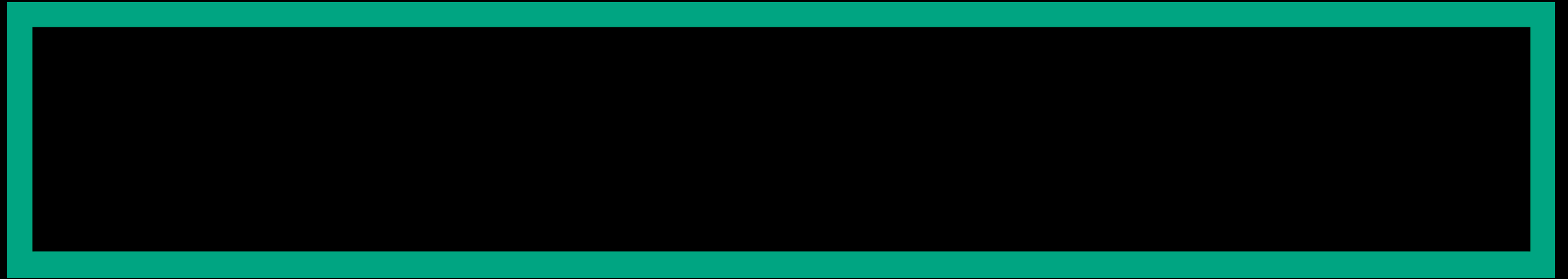


FIX_REGISTRY_COLLECTION

```
{
  "fix_commands": {
    "MinIO not installed": [
      "wget https://dl.min.io/server/minio/release/linux-amd64/minio -O /tmp/minio",
      "chmod +x /tmp/minio",
      "sudo mv /tmp/minio /usr/local/bin/minio",
      "sudo useradd -r minio-user -s /sbin/nologin || true",
      "sudo mkdir -p /data/minio",
      "sudo chown -R minio-user:minio-user /data/minio",
      "sudo tee /etc/systemd/system/minio.service > /dev/null << 'EOF'\n[Unit]\nDescription=MinIO Object Storage\nAfter=network.target\n\n[Service]\n\n[Install]\n\n\n[EOF]",
      "sudo systemctl daemon-reload",
      "sudo systemctl enable minio",
      "sudo systemctl start minio",
      "sudo apt install mc -y"
    ],
    "MinIO not running/crashed": [
      "sudo systemctl start minio",
      "sudo systemctl status minio --no-pager"
    ],
    "MinIO not on the right port": [
      "sudo systemctl stop minio",
      "sudo sed -i 's/--address.*:--address 0.0.0.0:9000 /g' /etc/systemd/system/minio.service || true",
      "sudo systemctl daemon-reload",
      "sudo systemctl start minio"
    ],
    "Firewall blocking MinIO port": [
      "sudo ufw allow 9000/tcp || true",
      "sudo ufw allow 9001/tcp || true",
      "sudo ufw reload || true",
      "sudo iptables -I INPUT -p tcp --dport 9000 -j ACCEPT || true"
    ]
  }
}
```

Iterative Autofix Workflow for MinIO (Alternative Approach)

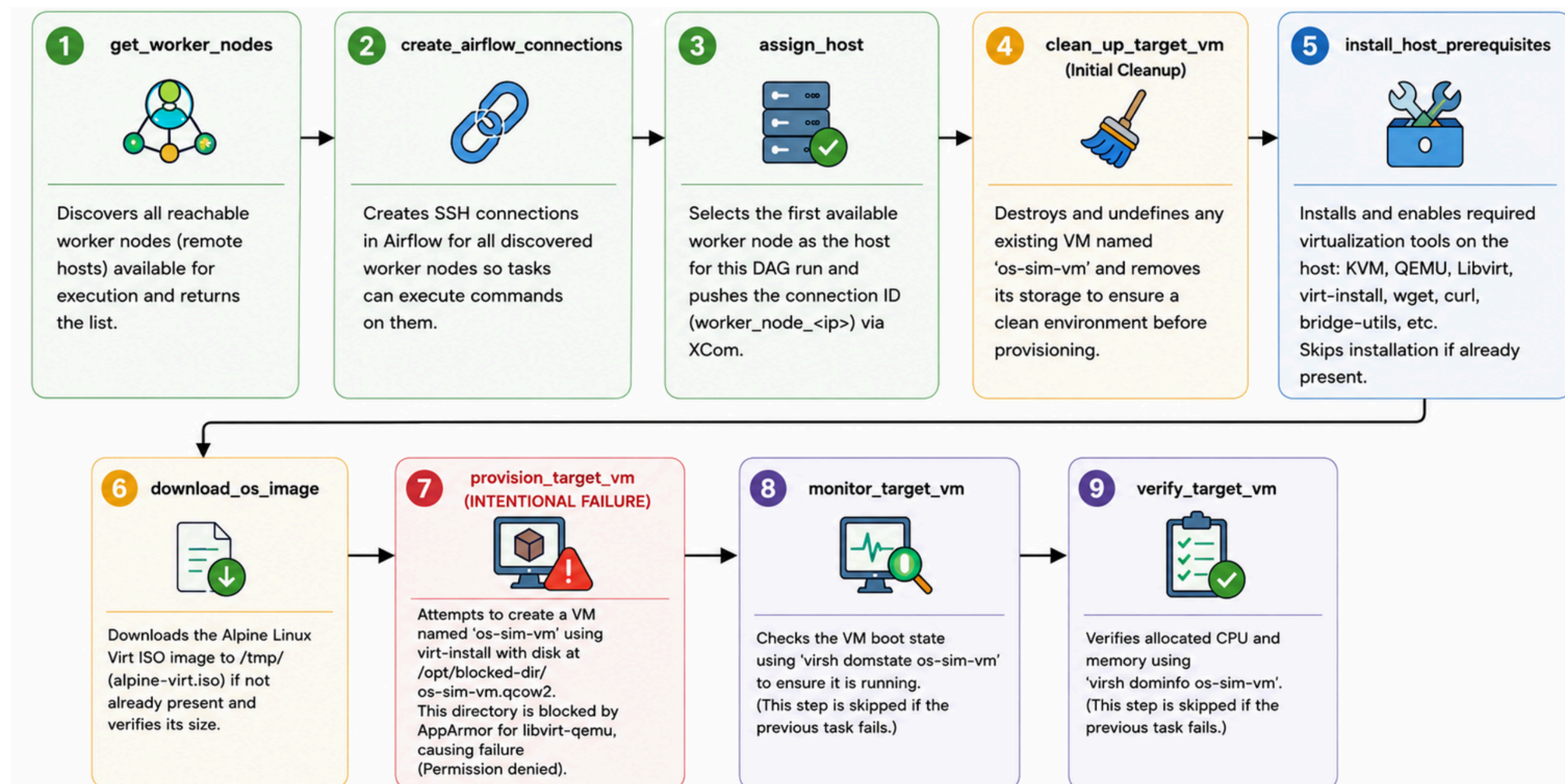




OS INSTALLATION

OS installation DAG

This DAG automates **OS provisioning** using **Apache Airflow, QEMU, and Libvirt**. It installs virtualization tools, downloads an **Alpine Linux image**, provisions a virtual machine, validates its execution, and performs cleanup. An intentional AppArmor **security policy violation** is injected during VM provisioning. The generated **failure logs are analyzed** by a RAG-powered multi-agent framework, which identifies the root cause, applies the appropriate remediation, validates the fix, and automatically re-executes the workflow



KEY LEARNINGS

- Designed an **Airflow-based infrastructure deployment** workflow to automate multi-step provisioning, validation, and monitoring tasks.
- Integrated a **LangGraph + RAG** pipeline to automatically analyze deployment failures and identify root causes from logs.
- Implemented **hybrid auto-remediation**, enabling both DAG-level fixes and SSH-based **OS-level recovery** on worker nodes.
- Built a **microservice architecture** using React, FastAPI, Airflow, ChromaDB, Redis, and Docker for scalable deployment automation.





THANK YOU

**Hewlett Packard
Enterprise**